



Documentación del Proyecto: Aplicación Acortador de URLs



Carrera:

Informática

Materia:

Desarrollo de aplicaciones móviles

Grupo:

2692

Alumnos:

David Jerónimo Rojas Avalos

José Octavio Escamilla Cuevas

Índice

1.	<i>Alcance del Proyecto</i>	4
2.	<i>Requerimientos Funcionales</i>	5
3.	<i>Diagrama de Datos (Firebase y Firestore)</i>	7
4.	<i>Diagrama de Secuencia</i>	8
4.1.	Secuencia de Inicio de Sesión con Google	8
4.2.	Secuencia de Actualización a Premium (Simulada)	9
4.3.	Secuencia de Acortamiento de URL (Completo)	10
5.	<i>Arquitectura del Backend (Servicio Web)</i>	11
5.1.	Descripción General	12
5.2.	Estructura de Archivos	12
5.3.	API RESTful	12
5.4.	Configuración de CORS	14
5.5.	Frontend Web	14
5.6.	Requisitos del Sistema y Despliegue	14
6.	<i>Diagrama de Datos (Backend MySQL)</i>	15
7.	<i>Diagrama de general de la arquitectura del sistema</i>	16
8.	<i>Componentes Principales de la App Android</i>	17
8.1	Modelos de Datos	17
8.2	Servicios API	18
9.	<i>Autenticación y Gestión de Usuarios</i>	19
9.1	Flujo de Autenticación	19
9.2	Modelo de Usuario	20
10.1	Acortamiento de URLs	20
10.2	Gestión de URLs (Historial)	21
10.3	Adapter para RecyclerView (UrlsAdapter.java)	22
11.	<i>Gestión de Suscripciones Premium</i>	22
11.1	PremiumActivity.java	22
12.	<i>Manejo de Errores</i>	23
13.	<i>Consideraciones de Seguridad</i>	24

14. Interfaces de Usuario (Layouts).....	24
14.1 Pantalla de Login (activity_login.xml)	24
14.2 Pantalla Principal (activity_main.xml)	25
14.3 Pantalla Premium (activity_premium.xml)	25
14.4 Pantalla de Historial (activity_url_list.xml)	26
14.5 Item de URL (item_url.xml)	26
14.6 Flujo de Navegación	26

1. Alcance del Proyecto

Este proyecto consiste en el desarrollo de una aplicación móvil para Android cuyo objetivo principal es permitir a los usuarios acortar URLs largas de manera sencilla. La aplicación integra servicios de autenticación de Google y una base de datos en la nube (Firebase Firestore) para gestionar los perfiles de usuario y las funcionalidades premium.

Características Clave:

- **Autenticación de Usuarios:** Permite a los usuarios iniciar y cerrar sesión utilizando su cuenta de Google.
- **Acortamiento de URLs:** Proporciona una interfaz para que los usuarios ingresen URLs largas y obtengan versiones acortadas a través de una API externa.
- **Gestión de Niveles de Usuario:** Implementa un sistema de niveles (gratis y premium) con límites de uso para usuarios gratuitos.
- **Simulación de Compra Premium:** Permite a los usuarios "actualizar" su cuenta a premium mediante una simulación de pago con tarjeta, desbloqueando funcionalidades ilimitadas.
- **Persistencia de Datos:** Almacena la información del perfil del usuario (estado premium, intentos restantes) en Firebase Firestore.
- **Actualización de UI en Tiempo Real:** La interfaz de usuario se actualiza dinámicamente para reflejar el estado actual del usuario (premium/gratuito, intentos restantes).

Límites del Alcance:

- La funcionalidad de pago premium es una **simulación** y no procesa transacciones de dinero real. No se integra con pasarelas de pago reales (Google Play Billing, Stripe, etc.).
- No se incluye la gestión de historial de URLs acortadas por usuario (aunque el XML sugiere un botón, la funcionalidad no se ha implementado).
- No se incluye la gestión de errores avanzados de red o de la API de acortamiento de URLs más allá de los mensajes básicos.
- No se incluye un backend personalizado más allá de Firebase Firestore y Authentication. La API de acortamiento de URLs se asume como un servicio externo.

2. Requerimientos Funcionales

El sistema debe cumplir con los siguientes requerimientos funcionales:

RF1: Autenticación de Usuarios

- **RF1.1:** El sistema debe permitir a los usuarios iniciar sesión utilizando su cuenta de Google.
- **RF1.2:** El sistema debe mostrar un mensaje de error si el inicio de sesión de Google falla.
- **RF1.3:** El sistema debe redirigir al usuario a la pantalla principal (MainActivity) tras un inicio de sesión exitoso.
- **RF1.4:** El sistema debe permitir a los usuarios cerrar sesión, redirigiéndolos a la pantalla de inicio de sesión.
- **RF1.5:** El sistema debe forzar la selección de cuenta de Google si el usuario ya tiene una sesión activa, pero se desea que elija.

RF2: Gestión de Perfil de Usuario

- **RF2.1:** Para nuevos usuarios, el sistema debe crear un documento de usuario en Firestore con su email, nombre de pantalla, estado isPremium como false y freeAttemptsRemaining con un valor inicial (ej. 5).
- **RF2.2:** El sistema debe cargar el perfil del usuario desde Firestore al iniciar la MainActivity.
- **RF2.3:** El sistema debe escuchar los cambios en tiempo real en el documento del usuario en Firestore y actualizar la interfaz de usuario de la MainActivity automáticamente.

RF3: Acortamiento de URLs

- **RF3.1:** El sistema debe permitir al usuario ingresar una URL larga en un campo de texto.
- **RF3.2:** El sistema debe validar que el campo de URL no esté vacío antes de intentar acortar.
- **RF3.3:** El sistema debe enviar la URL larga a una API externa de acortamiento de URLs.
- **RF3.4:** El sistema debe mostrar la URL acortada en un campo de texto visible.

- **RF3.5:** El sistema debe permitir al usuario copiar la URL acortada al portapapeles.
- **RF3.6:** El sistema debe mostrar mensajes de error si el acortamiento de URL falla (errores de red, errores de la API).

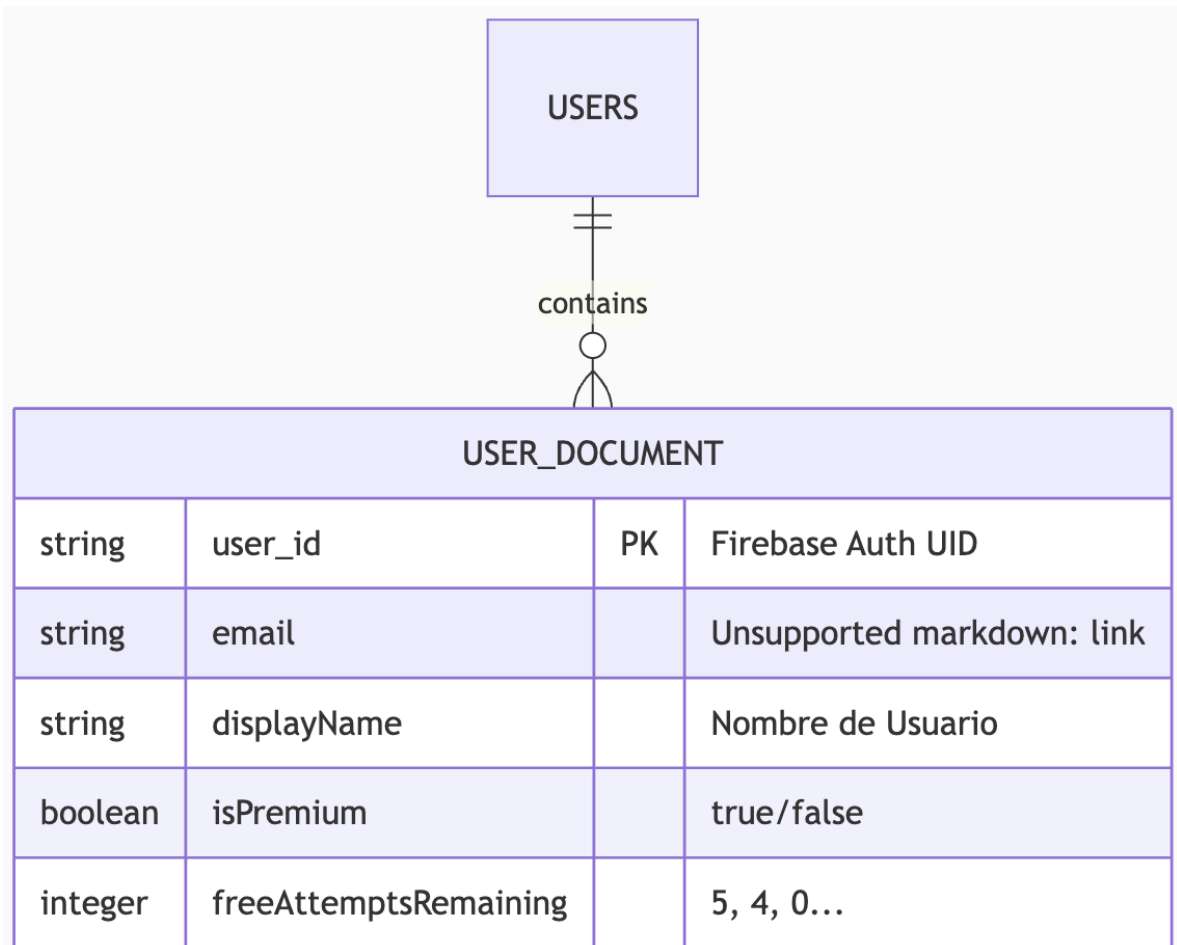
RF4: Funcionalidad Premium (Simulada)

- **RF4.1:** Los usuarios gratuitos deben tener un número limitado de freeAttemptsRemaining para acortar URLs.
- **RF4.2:** El sistema debe decrementar freeAttemptsRemaining en Firestore después de cada acortamiento exitoso para usuarios no premium.
- **RF4.3:** El sistema debe deshabilitar la función de acortamiento de URL si los freeAttemptsRemaining son 0 y el usuario no es premium.
- **RF4.4:** El sistema debe permitir al usuario navegar a una pantalla "Hazte Premium" (PremiumActivity).
- **RF4.5:** En la PremiumActivity, el sistema debe simular la entrada de datos de tarjeta de crédito (número, fecha, CVV, nombre).
- **RF4.6:** El sistema debe realizar validaciones básicas de formato para los campos de la tarjeta (ej. longitud del número, formato de fecha, CVV).
- **RF4.7:** Al "confirmar la compra" simulada, el sistema debe actualizar el campo isPremium del usuario a true en Firestore.
- **RF4.8:** Tras la "compra" simulada exitosa, el sistema debe mostrar un mensaje de confirmación y regresar a la MainActivity.

RF5: Actualización de Interfaz de Usuario (UI)

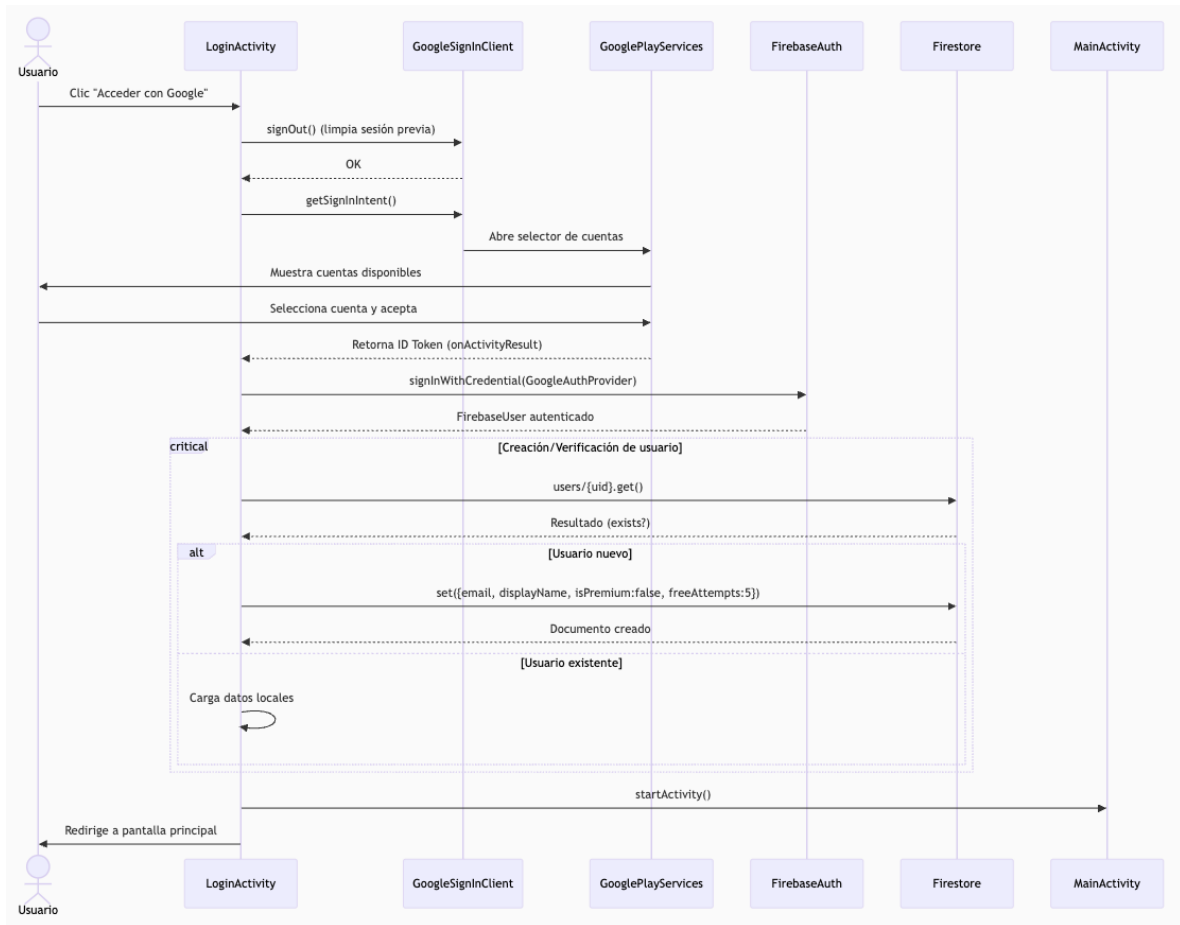
- **RF5.1:** La MainActivity debe mostrar el estado actual del usuario (ej. "Intentos Gratis: X" o "¡Usuario Premium!").
- **RF5.2:** La MainActivity debe ocultar el botón "Hazte Premium" y habilitar el acortamiento ilimitado si el usuario es premium.
- **RF5.3:** La MainActivity debe mostrar el botón "Hazte Premium" y gestionar el estado de habilitación del acortador si el usuario no es premium.

3. Diagrama de Datos (Firebase y Firestore)

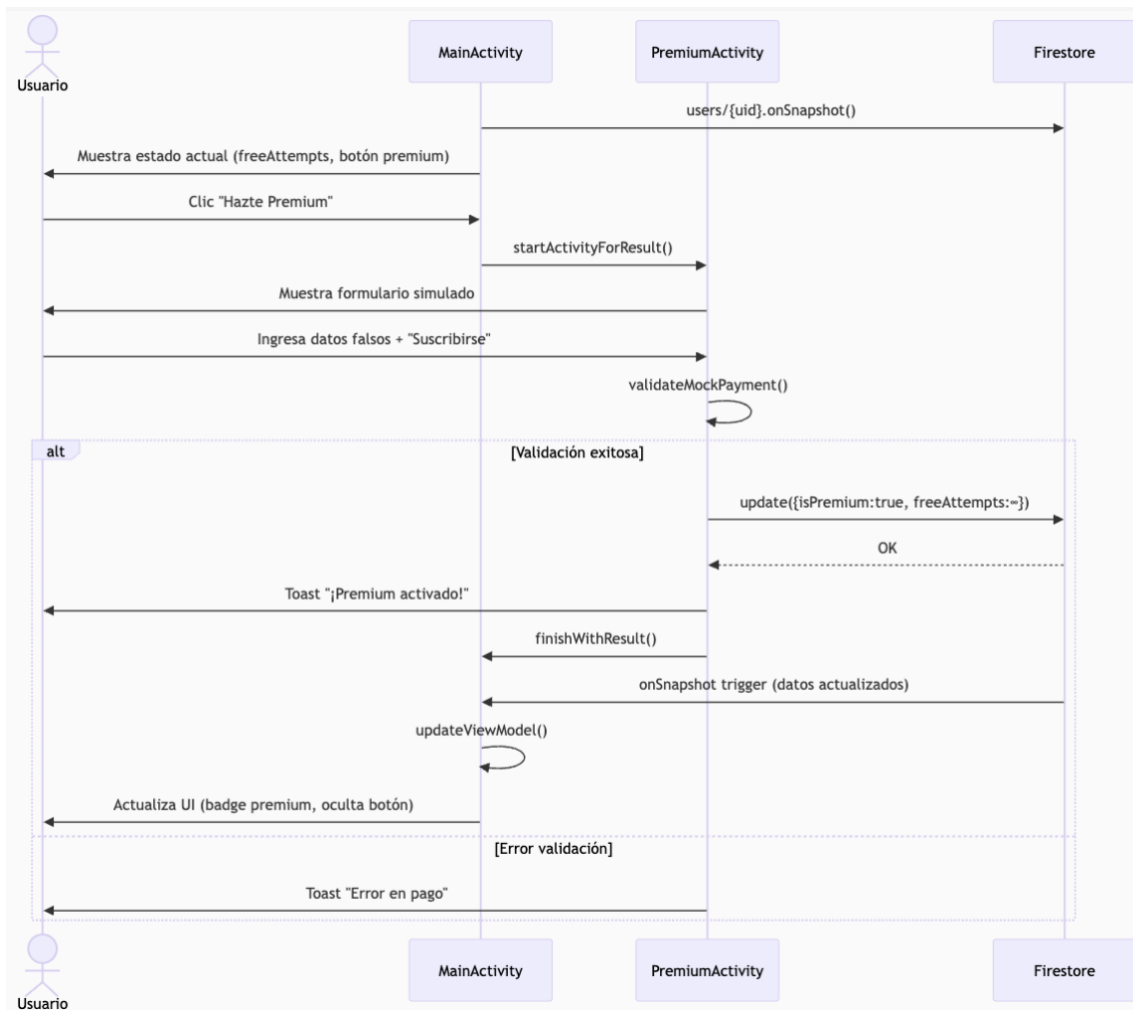


4. Diagrama de Secuencia

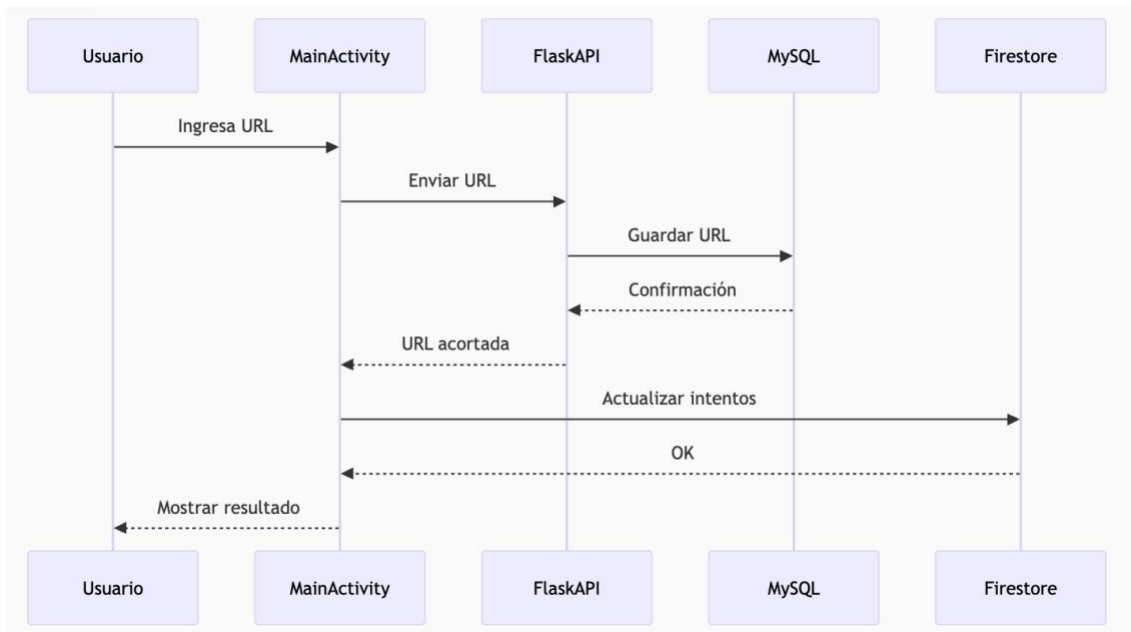
4.1. Secuencia de Inicio de Sesión con Google



4.2. Secuencia de Actualización a Premium (Simulada)

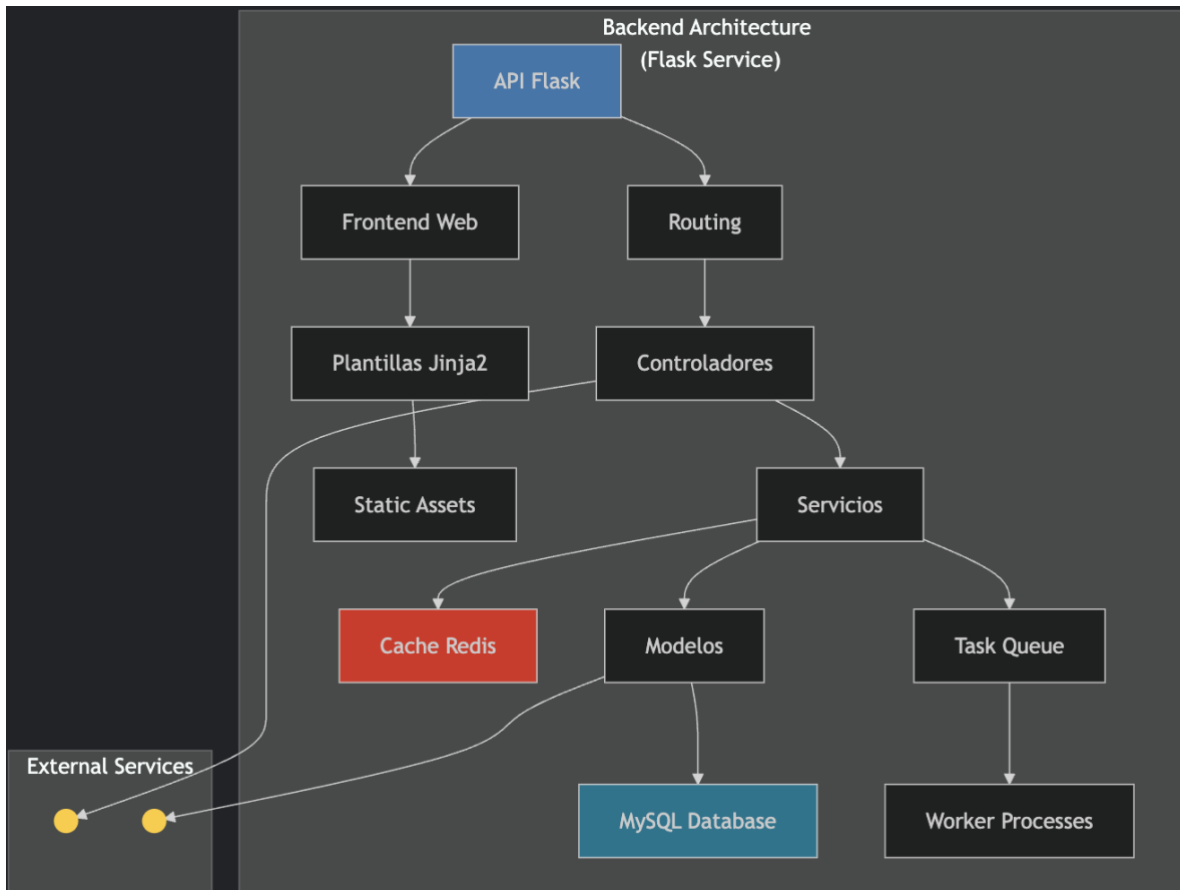


4.3. Secuencia de Acortamiento de URL (Completo)



5. Arquitectura del Backend (Servicio Web)

Este proyecto incluye un servicio web (backend) que gestiona la lógica de acortamiento de URLs y la persistencia de los datos de las URLs en una base de datos MySQL. También sirve un frontend web interactivo.



Este proyecto incluye un servicio web (backend) que gestiona la lógica de acortamiento de URLs y la persistencia de los datos de las URLs en una base de datos MySQL. También sirve un frontend web interactivo.

5.1. Descripción General

El backend es un servicio web desarrollado en **Python con Flask**, que proporciona una **API RESTful** para la integración con la aplicación móvil Android y un **frontend web interactivo** para usuarios de navegador. Utiliza **MySQL** para el almacenamiento persistente de las URLs acortadas.

5.2. Estructura de Archivos

La estructura del directorio del backend es la siguiente:

```
/acortador-url/  
├── AcortadorURL.py    # Backend Flask (API y frontend)  
├── requirements.txt   # Dependencias de Python  
├── static/  
│   ├── style.css     # Estilos CSS del frontend  
│   └── script.js     # Lógica JavaScript del frontend  
└── templates/  
    └── index.html    # Página principal del frontend
```

5.3. API RESTful

La API RESTful es el punto de integración principal para la aplicación Android.

5.3.1. Acortar una URL

Método: POST

- **Ruta:** /api/shorten
- **Content-Type:** application/json
- **Parámetros (JSON):**
 - {
 - "url": "https://ejemplo.com/url-muy-larga"
 - }
-

Respuesta exitosa (200 OK):

- {
- "original_url": "https://ejemplo.com/url-muy-larga",
- "short_url": "http://fcau.up.railway.app/6Ji5BQ"
- }
-

Errores posibles:

- 400 Bad Request: Si no se proporciona la URL.
- 500 Internal Server Error: Errores internos del servidor.

5.3.2. Listar URLs acortadas

Método: GET

Ruta: /api/urls

Respuesta exitosa (200 OK):

```
[
  {
    "original_url": "https://ejemplo.com/url1",
    "short_url": "http://fcau.up.railway.app/6Ji5BQ",
    "created_at": "2023-07-20T12:34:56",
    "clicks": 5
  },
  {
    "original_url": "https://ejemplo.com/url2",
    "short_url": "http://fcau.up.railway.app/6Ji5BQ",
    "created_at": "2023-07-19T10:11:12",
    "clicks": 2
  }
]
```

5.3.3. Eliminar una URL

Método: DELETE

Ruta: /api/urls/<short_code>

Respuesta exitosa (200 OK):

```
{
  "message": "URL eliminada correctamente"
}
```

Errores posibles:

- 404 Not Found: Si el código corto no existe.
- 500 Internal Server Error: Errores internos del servidor.

5.3.4. Redirección

Método: GET

Ruta: /<short_code>

Comportamiento:

- Redirige a la URL original correspondiente.
- Incrementa el contador de clics automáticamente.

5.4. Configuración de CORS

La API está configurada para aceptar solicitudes desde aplicaciones Android y otros orígenes con los siguientes parámetros CORS:

- **Orígenes permitidos:** Cualquier origen (*)
- **Métodos permitidos:** GET, POST, DELETE, OPTIONS
- **Headers permitidos:** Content-Type

5.5. Frontend Web

El backend también sirve un frontend web interactivo con las siguientes características y funcionalidades:

- **Características:**
 1. Formulario para acortar URLs.
 2. Visualización del resultado con opción de copiar.
 3. Historial de URLs acortadas recientemente.
 4. Funcionalidad para eliminar URLs.
 5. Diseño responsivo y amigable.
- **Funcionalidades JavaScript:**
 - `loadUrlHistory()`: Carga y muestra el historial de URLs.
 - `copyToClipboard(text)`: Copia una URL al portapapeles.
 - `deleteUrl(shortCode)`: Elimina una URL acortada.
 - `truncateText(text, maxLength)`: Acorta texto largo para visualización.

5.6. Requisitos del Sistema y Despliegue

5.6.1. Dependencias (*requirements.txt*)

```
flask==2.3.2
mysql-connector-python==8.0.33
flask-cors==3.0.10
gunicorn==20.1.0
```

5.6.2. Base de Datos MySQL

El sistema utiliza MySQL con la siguiente estructura de tabla para almacenar las URLs:

```
CREATE TABLE IF NOT EXISTS urls (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  original_url TEXT NOT NULL,  
  short_code VARCHAR(8) NOT NULL UNIQUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  clicks INT DEFAULT 0  
)
```

5.6.3. Variables de Entorno para Configuración

- **MYSQLHOST:** Servidor MySQL (default: localhost)
- **MYSQLUSER:** Usuario MySQL (default: root)
- **MYSQLPASSWORD:** Contraseña MySQL (default: vacía)
- **MYSQLDATABASE:** Nombre de la base de datos (default: url_shortener)
- **MYSQLPORT:** Puerto MySQL (default: 3306)
- **PORT:** Puerto para la aplicación Flask (default: 5000)

5.6.4. Despliegue

1. **Instalar dependencias:** `pip install -r requirements.txt`
2. **Configurar variables de entorno** para la base de datos.
3. **Ejecutar la aplicación:** `python AcortadorURL.py`
4. **(Opcional) Para producción, usar Gunicorn:** `gunicorn -w 4 -b 0.0.0.0:5000 AcortadorURL:app`

5.6.5. Health Check

- **Endpoint:** `/health`
- **Respuesta:**
- `{"status": "healthy"}`
-

6. Diagrama de Datos (Backend MySQL)

El diagrama de datos muestra la estructura de la tabla `urls` en la base de datos MySQL del backend.

```
erDiagram  
  urls {  
    INT id PK  
    TEXT original_url
```

```

    VARCHAR(8) short_code UK
    TIMESTAMP created_at
    INT clicks
}

```

7. Diagrama de general de la arquitectura del sistema

Este diagrama muestra los componentes principales de todo el sistema (aplicación Android y servicio web) y cómo interactúan entre sí y con los servicios externos.

```

graph TD
    subgraph Android_App [Android App]
        A[UI: LoginActivity] --> B[UI: MainActivity]
        B --> C[UI: PremiumActivity]
        B --> D[UI: UrlListActivity (Historial)]

        B -- Gestiona --> E[Modelo de Datos: User.java]
        C -- Gestiona --> E

        F[Firebase Auth SDK -- Autenticación] --> G[Firebase Authentication]
        H[Firestore SDK -- Lectura/Escritura] --> I[Firestore Database]
        J[Retrofit Client] -- Peticiones API REST --> K[Backend: Servicio Web Flask]
    end

    subgraph Backend [Backend: Servicio Web Flask]
        K -- Sirve --> L[Frontend Web (HTML/CSS/JS)]
        K -- Interactúa --> M[Base de Datos MySQL]
    end

    G -- Provee UID/Email --> I
    A -- Usa --> F
    B -- Usa --> F
    B -- Usa --> H
    C -- Usa --> F
    C -- Usa --> H
    B -- Usa --> J
    D -- Usa --> J
    K -- Almacena/Consulta --> M

```

Explicación de los Componentes:

- **Android App:** La aplicación móvil desarrollada en Java.
 - **UI (LoginActivity, MainActivity, PremiumActivity, UrlListActivity):** Las diferentes pantallas con las que interactúa el usuario.
 - **Modelo de Datos (User.java):** La clase Java que define la estructura de los datos del usuario almacenados en Firestore.
 - **Firebase Auth SDK:** La librería de Firebase para gestionar la autenticación de usuarios (incluyendo Google Sign-In).

- **Firestore SDK:** La librería de Firebase para interactuar con la base de datos Firestore (leer, escribir, escuchar cambios en tiempo real).
 - **Retrofit Client:** Una librería para realizar peticiones HTTP a APIs REST, utilizada aquí para comunicarse con el Backend.
- **Backend: Servicio Web Flask:** El servicio web desarrollado en Python con Flask.
 - **Frontend Web (HTML/CSS/JS):** La interfaz de usuario web servida por el backend.
 - **Base de Datos MySQL:** La base de datos relacional utilizada por el backend para almacenar las URLs acortadas.
- **Firestore Authentication:** Servicio de Firebase para gestionar usuarios y sus credenciales.
- **Firestore Database:** Base de datos NoSQL de Firebase para almacenar los perfiles de usuario de la aplicación Android.

8. Componentes Principales de la App Android

Esta sección detalla los modelos de datos y los servicios API utilizados en la aplicación Android.

8.1 Modelos de Datos

User.java

```
public class User {
    private String email;
    private String displayName;
    private boolean isPremium;
    private int freeAttemptsRemaining;
    // Getters y setters
}
```

Propósitos:

- Almacena información del usuario autenticado.
- Controla el estado premium y los intentos gratuitos.
- Se sincroniza con Firestore.

ShortenRequest.java

```
public class ShortenRequest {
    private String url;

    public ShortenRequest(String url) {
        this.url = url;
    }

    public String getUrl() {
        return url;
    }
}
```

ShortenResponse.java

import com.google.gson.annotations.SerializedName; // Necesario si los nombres de campo JSON son diferentes

```
public class ShortenResponse {
    @SerializedName("original_url")
    private String originalUrl;
    @SerializedName("short_url")
    private String shortUrl;
    @SerializedName("short_code")
    private String shortCode;
    @SerializedName("created_at")
    private String createdAt;
    private int clicks;

    // Constructor vacío requerido por Gson para deserialización
    public ShortenResponse() {}

    // Getters
    public String getOriginalUrl() { return originalUrl; }
    public String getShortUrl() { return shortUrl; }
    public String getShortCode() { return shortCode; }
    public String getCreatedAt() { return createdAt; }
    public int getClicks() { return clicks; }

    // Setters (opcionales si solo se usa para deserialización, pero buena práctica)
    public void setOriginalUrl(String originalUrl) { this.originalUrl = originalUrl; }
    public void setShortUrl(String shortUrl) { this.shortUrl = shortUrl; }
    public void setShortCode(String shortCode) { this.shortCode = shortCode; }
    public void setCreatedAt(String createdAt) { this.createdAt = createdAt; }
    public void setClicks(int clicks) { this.clicks = clicks; }
}
```

8.2 Servicios API

ApiService.java (Retrofit Interface)

```
import retrofit2.Call;
import retrofit2.http.Body;
import retrofit2.http.DELETE;
import retrofit2.http.GET;
import retrofit2.http.POST;
import retrofit2.http.Path;
import java.util.List;

public interface ApiService {
    @POST("/api/shorten")
    Call<ShortenResponse> shortenUrl(@Body ShortenRequest request);

    @GET("/api/urls")
    Call<List<ShortenResponse>> getUrls();

    @DELETE("/api/urls/{shortCode}")
    Call<Void> deleteUrl(@Path("shortCode") String shortCode);
}
```

RetrofitClient.java

```
import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;

public class RetrofitClient {
    public static final String BASE_URL = "https://fcau.up.railway.app/"; // URL base del backend
    desplegado
    private static Retrofit retrofit = null;

    public static ApiService getApiService() {
        if (retrofit == null) {
            retrofit = new Retrofit.Builder()
                .baseUrl(BASE_URL)
                .addConverterFactory(GsonConverterFactory.create())
                .build();
        }
        return retrofit.create(ApiService.class);
    }
}
```

Configuración clave:

- URL base del backend desplegado.
- Conversión automática JSON ↔ Objetos Java con Gson.
- Patrón Singleton para la instancia Retrofit.

9. Autenticación y Gestión de Usuarios

9.1 Flujo de Autenticación

1. **LoginActivity:** Maneja el inicio de sesión con Google.
2. **Firebase Auth:** Autenticación principal.
3. **Firestore:** Almacena datos adicionales del usuario.

LoginActivity.java (Fragmentos clave)

```
// Configuración de Google Sign-In
GoogleSignInOptions gso =
GoogleSignInOptions.Builder(GoogleSignInOptions.DEFAULT_SIGN_IN)
    .requestIdToken(getString(R.string.default_web_client_id))
    .requestEmail()
    .build();

// Autenticación con Firebase
private void firebaseAuthWithGoogle(String idToken) {
    AuthCredential credential = GoogleAuthProvider.getCredential(idToken, null);
    mAuth.signInWithCredential(credential)
        .addOnCompleteListener(this, task -> {
            if (task.isSuccessful()) {
                FirebaseUser user = mAuth.getCurrentUser();
                checkAndCreateUserDocumentInFirestore(user);
            }
        });
}
```

```

    }
    });
}

// Creación/verificación de usuario en Firestore
private void checkAndCreateUserDocumentInFirestore(FirebaseUser firebaseUser) {
    DocumentReference userRef = db.collection("users").document(firebaseUser.getId());
    userRef.get().addOnCompleteListener(task -> {
        if (!task.getResult().exists()) {
            User newUser = new User(email, displayName, false, 5);
            userRef.set(newUser);
        }
    });
}
}

```

9.2 Modelo de Usuario

- **Usuarios gratuitos:** 5 intentos iniciales.
- **Usuarios premium:** Intentos ilimitados.
- **Datos almacenados:**
 - Email
 - Nombre para mostrar
 - Estado premium (isPremium)
 - Intentos restantes (freeAttemptsRemaining)

10. Funcionalidades Principales

10.1 Acortamiento de URLs

Flujo en MainActivity.java:

Validación de entrada:

```

1. private boolean isValidUrl(String url) {
2.     String urlPattern = "^((https?:/)?(\\w-|+\\.)+([a-z\\u00a1-\\uffff]{2,63})" +
3.         "(/[-\\w\\u00a1-\\uffff@:%_+~#?&/=]*)?$";
4.     Pattern pattern = Pattern.compile(urlPattern, Pattern.CASE_INSENSITIVE);
5.     return pattern.matcher(url).matches();
6. }
7.

```

Llamada a la API:

```

8. ShortenRequest request = new ShortenRequest(originalUrl);
9. RetrofitClient.getApiService().shortenUrl(request).enqueue(new
    Callback<ShortenResponse>() {
10.     @Override

```

```

11. public void onResponse(Call<ShortenResponse> call, Response<ShortenResponse>
    response) {
12.     if (response.isSuccessful()) {
13.         String shortUrl = response.body().getShortUrl();
14.         tvResult.setText("URL Acortada: " + shortUrl);
15.         if (!currentUserModel.isPremium()) {
16.             decrementAttempts();
17.         }
18.     }
19. }
20. });
21.

```

Actualización de intentos:

```

22. private void decrementAttempts() {
23.     int newAttempts = currentUserModel.getFreeAttemptsRemaining() - 1;
24.     currentUserModel.setFreeAttemptsRemaining(newAttempts);
25.     db.collection("users").document(mAuth.getCurrentUser().getUid())
26.         .update("freeAttemptsRemaining", newAttempts);
27. }
28.

```

10.2 Gestión de URLs (Historial)

Listado de URLs (*UrlListActivity.java*)

import java.util.List; // Asegúrate de importar List

```

private void loadUrls() {
    RetrofitClient.getApiService().getUrls().enqueue(new Callback<List<ShortenResponse>>() {
        @Override
        public void onResponse(Call<List<ShortenResponse>> call,
Response<List<ShortenResponse>> response) {
            if (response.isSuccessful()) {
                // urlList es una lista de ShortenResponse en tu Activity/Adapter
                // adapter es tu RecyclerView.Adapter
                // Asegúrate de que urlList y adapter estén inicializados
                // urlList.clear();
                // urlList.addAll(response.body());
                // adapter.notifyDataSetChanged();
            }
        }
    });
}

```

Eliminación de URLs

```

private void deleteUrl(String shortCode) {
    RetrofitClient.getApiService().deleteUrl(shortCode).enqueue(new Callback<Void>() {
        @Override
        public void onResponse(Call<Void> call, Response<Void> response) {
            if (response.isSuccessful()) {

```

```

        // adapter.removeItem(shortCode); // Método para eliminar del adapter
    }
}
});
}

```

10.3 Adapter para RecyclerView (UrlsAdapter.java)

Funcionalidades:

- Muestra lista de URLs acortadas.
- Botones para copiar y eliminar.
- Actualización dinámica de la lista.

```

// Ejemplo de onBindViewHolder en UrlsAdapter.java
import androidx.annotation.NonNull; // Importa si no está presente

public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
    ShortenResponse url = urls.get(position);
    holder.tvOriginalUrl.setText(url.getOriginalUrl());
    holder.tvShortUrl.setText(url.getShortUrl());

    holder.btnCopy.setOnClickListener(v -> {
        // Lógica para copiar al portapapeles (ej. usando ClipboardManager)
    });

    holder.btnDelete.setOnClickListener(v -> {
        // deleteListener es una interfaz definida en tu adapter para comunicar la acción a la Activity
        // deleteListener.onDeleteClick(url.getShortCode());
    });
}

```

11. Gestión de Suscripciones Premium

11.1 PremiumActivity.java

Validaciones:

1. Número de tarjeta (algoritmo de Luhn).
2. Fecha de expiración.
3. CVV.
4. Nombre del titular.

```

private boolean isValidCardNumber(String cardNumber) {
    // Implementación del algoritmo de Luhn
    String cleanedNumber = cardNumber.replaceAll("[^0-9]", "");
    if (cleanedNumber.length() != 16) { return false; }
    int sum = 0;
    boolean alternate = false;

```

```

for (int i = cleanedNumber.length() - 1; i >= 0; i--) {
    int digit = Integer.parseInt(cleanedNumber.substring(i, i + 1));
    if (alternate) {
        digit *= 2;
        if (digit > 9) digit = (digit % 10) + 1;
    }
    sum += digit;
    alternate = !alternate;
}
return (sum % 10 == 0);
}

// Otros métodos de validación (isValidExpiryDate, isValidCvv, isValidCardHolderName)
// ...

// Actualización a Premium:
private void updateUserToPremium() {
    FirebaseUser currentUser = mAuth.getCurrentUser();
    if (currentUser != null) {
        db.collection("users").document(currentUser.getId())
            .update("isPremium", true)
            .addOnSuccessListener(aVoid -> {
                Toast.makeText(this, "¡Felicidades! Ahora eres Premium.",
                    Toast.LENGTH_LONG).show();
                finish();
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error al actualizar usuario a premium: ", e);
                Toast.makeText(this, "Error al procesar suscripción. Inténtalo de nuevo.",
                    Toast.LENGTH_SHORT).show();
            });
    }
}

```

12. Manejo de Errores

Ejemplos de manejo de errores en las llamadas API:

```

import java.io.IOException; // Importa si no está presente

RetrofitClient.getApiService().shortenUrl(request).enqueue(new Callback<ShortenResponse>() {
    @Override
    public void onFailure(Call<ShortenResponse> call, Throwable t) {
        Toast.makeText(MainActivity.this,
            "Error de red: " + t.getMessage(), Toast.LENGTH_LONG).show();
    }
})

@Override
public void onResponse(Call<ShortenResponse> call, Response<ShortenResponse> response) {
    if (!response.isSuccessful()) {
        String errorMsg = "Error: " + response.code();
        try {
            errorMsg += " - " + response.errorBody().string();
        } catch (IOException e) {
            // Ignorar error de lectura del cuerpo de la respuesta
        }
    }
}

```

```

    } catch (IOException e) {
        e.printStackTrace();
    }
    Toast.makeText(MainActivity.this, errorMsg, Toast.LENGTH_LONG).show();
}
}
});

```

13. Consideraciones de Seguridad

1. **Autenticación:** Google Sign-In + Firebase Auth para una autenticación segura de usuarios.
2. **Validación:**
 - URLs antes de procesar (en el cliente y en el backend).
 - Datos de tarjeta en simulaciones (solo formato, no seguridad real).
3. **Firestore Rules:** Deben configurarse reglas de seguridad en Firestore para que los usuarios solo accedan a sus propios datos de perfil.
4. **HTTPS:** Todas las comunicaciones (entre la app y Firebase, y entre la app y el backend) son sobre HTTPS para garantizar la encriptación.
5. **Backend (MySQL):** El backend debe implementar medidas de seguridad adecuadas para la base de datos MySQL (ej. conexiones seguras, saneamiento de entradas, prevención de inyección SQL).

14. Interfaces de Usuario (Layouts)

14.1 Pantalla de Login (activity_login.xml)

Propósito: Permite a los usuarios iniciar sesión con su cuenta de Google.
Componentes principales:

- LinearLayout vertical centrado.
- TextView con el título de la aplicación.
- Botón oficial de Google Sign-In (com.google.android.gms.common.SignInButton).

Características de diseño:

- Diseño minimalista y centrado.
- Padding de 16dp en todos los lados.
- Margen inferior de 24dp para el título.

14.2 Pantalla Principal (activity_main.xml)

Propósito: Interfaz principal para acortar URLs y gestionar la cuenta.

Componentes principales:

- ScrollView para contenido desplazable.
- CardView con esquinas redondeadas para el formulario.
- EditText para ingresar la URL.
- Botón "Acortar URL".
- TextView para mostrar resultados.
- Botón "Copiar URL" (inicialmente oculto).
- Botón "Ver Historial".
- Contador de intentos gratuitos.
- Botón "Hazte Premium".
- Botón "Cerrar Sesión". **Características de diseño:**
- Fondo gris claro (#F2F2F2).
- Elevación de 6dp para el CardView.
- Esquinas redondeadas de 8dp.
- Colores principales:
 - Morado (#6200EE) para botones principales.
 - Verde (#4CAF50) para acciones positivas.
 - Rojo (#D32F2F) para cerrar sesión.
 - Naranja (#FF9800) para premium.

14.3 Pantalla Premium (activity_premium.xml)

Propósito: Permite a los usuarios suscribirse al servicio premium.

Componentes principales:

- ScrollView para formulario largo.
- EditTexts para:
 - Número de tarjeta (16 dígitos).
 - Fecha de vencimiento (MM/AA).
 - CVV (3 dígitos).
 - Nombre del titular.
- Botón "Suscribirse a Premium".
- Nota sobre simulación de pago. **Características de diseño:**
- Fondo blanco (#FFFDFD).
- EditTexts con fondo redondeado (@drawable/rounded_edittext).
- Validación visual mediante inputType:
 - number para tarjeta y CVV.
 - datetime para fecha.
 - textPersonName para nombre.
- AutofillHints para ayuda en autocompletado.

14.4 Pantalla de Historial (`activity_url_list.xml`)

Propósito: Muestra el historial de URLs acortadas por el usuario. **Componentes principales:**

- Botón "Volver".
- TextView de título.
- RecyclerView para lista de URLs. **Características de diseño:**
- Diseño simple y funcional.
- Padding de 16dp en todos los lados.
- RecyclerView ocupa todo el espacio disponible.

14.5 Ítem de URL (`item_url.xml`)

Propósito: Diseño de cada elemento en la lista del historial. **Componentes principales:**

- CardView con elevación de 4dp.
- TextView para URL original (2 líneas máximo).
- TextView para URL cortada (color morado, autoLink web).
- TextView para contador de clics.
- Botones "Copiar" y "Eliminar". **Características de diseño:**
- Margen de 8dp alrededor del CardView.
- Esquinas redondeadas de 8dp.
- Botones alineados a la derecha.
- Color rojo (#FF3D00) para botón eliminar.
- Texto truncado con ellipsis para URLs largas.

14.6 Flujo de Navegación

1. **LoginActivity:** Primera pantalla, solo acceso con Google.
2. **MainActivity:** Pantalla principal después del login.
 - Acceso a:
 - PremiumActivity (mediante btnGoPremium).
 - UrlListActivity (mediante btnViewHistory).
3. **PremiumActivity:** Suscripción premium (retorna a MainActivity).
4. **UrlListActivity:** Historial de URLs (btnBack retorna a MainActivity).